

Introduction to NLP

Tasks, eras, breakthroughs, and the journey ahead

Dr. A. Belcaid

Table of contents

1 Your route through the course **2**
1.1 Where everything lives . . . 3
1.2 Assessment at a glance . . . 3

2 What can machines do with language? **4**
2.1 One message, many possible tasks . . . 5

3 One problem, three eras **6**
3.1 Era I · Tell the machine the rules . . . 7
3.2 Era II · Learn patterns from examples . . . 7
3.3 Era III · Learn representations and reuse knowledge . . . 8

4 Breakthroughs that changed the field **9**
4.1 The recurring pattern . . . 9
4.1.1 Statistical language models . . . 10
4.1.2 Distributed representations . . . 10
4.1.3 Neural sequence models . . . 10
4.1.4 Attention . . . 10
4.1.5 Transformers . . . 10
4.1.6 Pretraining and instruction following . . . 10
4.1.7 Retrieval and tools . . . 10

5 The questions that follow us **11**

6 Next: how does text enter a model? 11



Figure 1: From hand-written rules to statistical learning, neural networks, and multilingual language systems

Natural language processing sits at a remarkable intersection: **human communication**, **computational models**, and **real-world decisions**. This course is about understanding that intersection well enough to build useful systems—and to recognize where those systems succeed, fail, or mislead.

We begin with the map rather than the machinery. Today we will see what NLP systems do, compare three ways of building them, and trace the breakthroughs that produced modern language models. In the next chapter, we will slow down and ask how written language becomes something a model can process.

> **By the end of this session**
>
> You should be able to:
>
> 1. navigate the course and explain how its lectures, notes, and projects fit together;
> 2. recognize the major families of NLP tasks;
> 3. contrast rule-based, statistical, and neural approaches;
> 4. place Transformers and language models in a longer history of NLP; and
> 5. explain why tokenization is the necessary next step.

1 Your route through the course

The website is the course's home base. Each week begins with a motivating question, develops the underlying ideas, and ends with a system you can run, inspect, and improve.

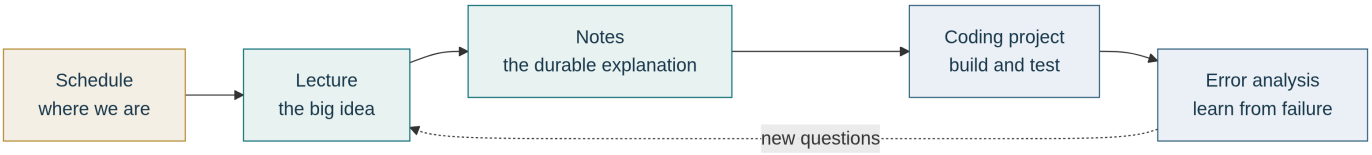


Figure 2: The weekly learning loop: ideas become experiments, and experiments create better questions.

1.1 Where everything lives

Website area	Use it for	Your habit
Schedule	The current topic, notes, readings, and project	Start here every week
Lecture	Motivation, demonstrations, and discussion	Participate and ask questions
Notes	Explanations, examples, diagrams, and references	Return while implementing
Coding projects	Focused practice with immediate feedback	Build, test, then analyze errors
Student projects	Final-project themes, milestones, and presentations	Connect course ideas to a problem you care about

> **! Projects are the center of the course**
>
> Knowing the vocabulary is not enough. You will learn NLP by making systems behave, investigating why they fail, and communicating what you discovered.

1.2 Assessment at a glance

The intended balance keeps most of the grade on sustained, reproducible work.

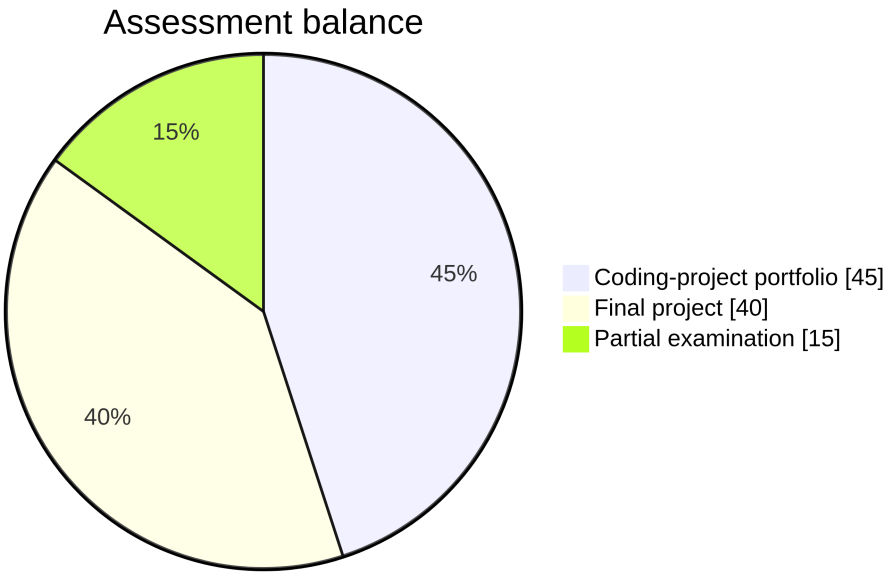


Figure 3: Proposed assessment balance; final percentages will be confirmed on the logistics page.

The **coding-project portfolio** rewards consistent implementation and reflection. The **final project** asks you to formulate a problem, choose a defensible approach, evaluate it, and present the result. A short **partial examination** checks that you can connect the central ideas without relying on code.

2 What can machines do with language?

NLP is not one task. It is a family of problems connected by a shared input or output: human language.

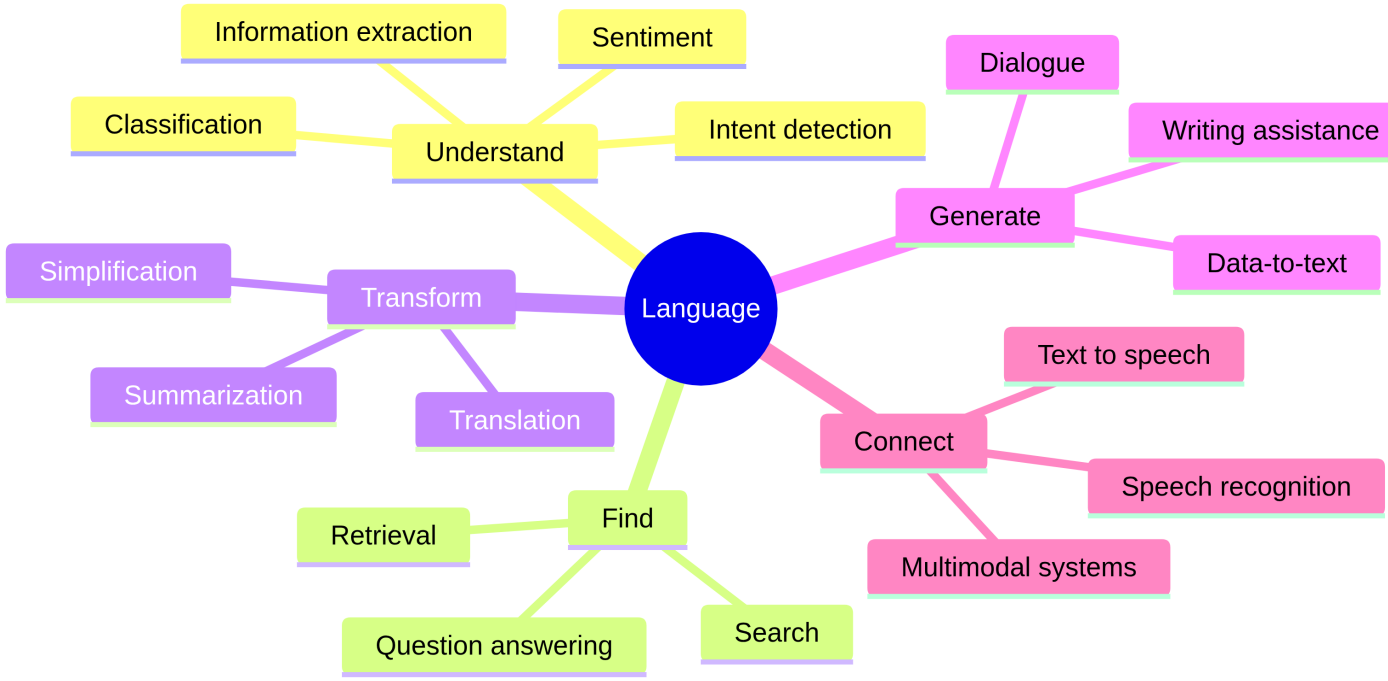


Figure 4: A task map for NLP. Many real applications combine several branches.

2.1 One message, many possible tasks

Consider the message:

> The battery lasts all day, but the camera struggles at night.

Different NLP systems could ask very different questions:

Task	Possible output
Sentiment analysis	Mixed
Aspect extraction	`battery`, `camera`
Aspect sentiment	Battery: positive; camera: negative
Summarization	Good battery, weak low-light camera
Translation	The same meaning in another language
Retrieval	Products with stronger night photography
Dialogue	A follow-up question about the user's priorities

The text is identical, but the **task determines what information matters**. There is no universally correct representation of language independent of a goal.

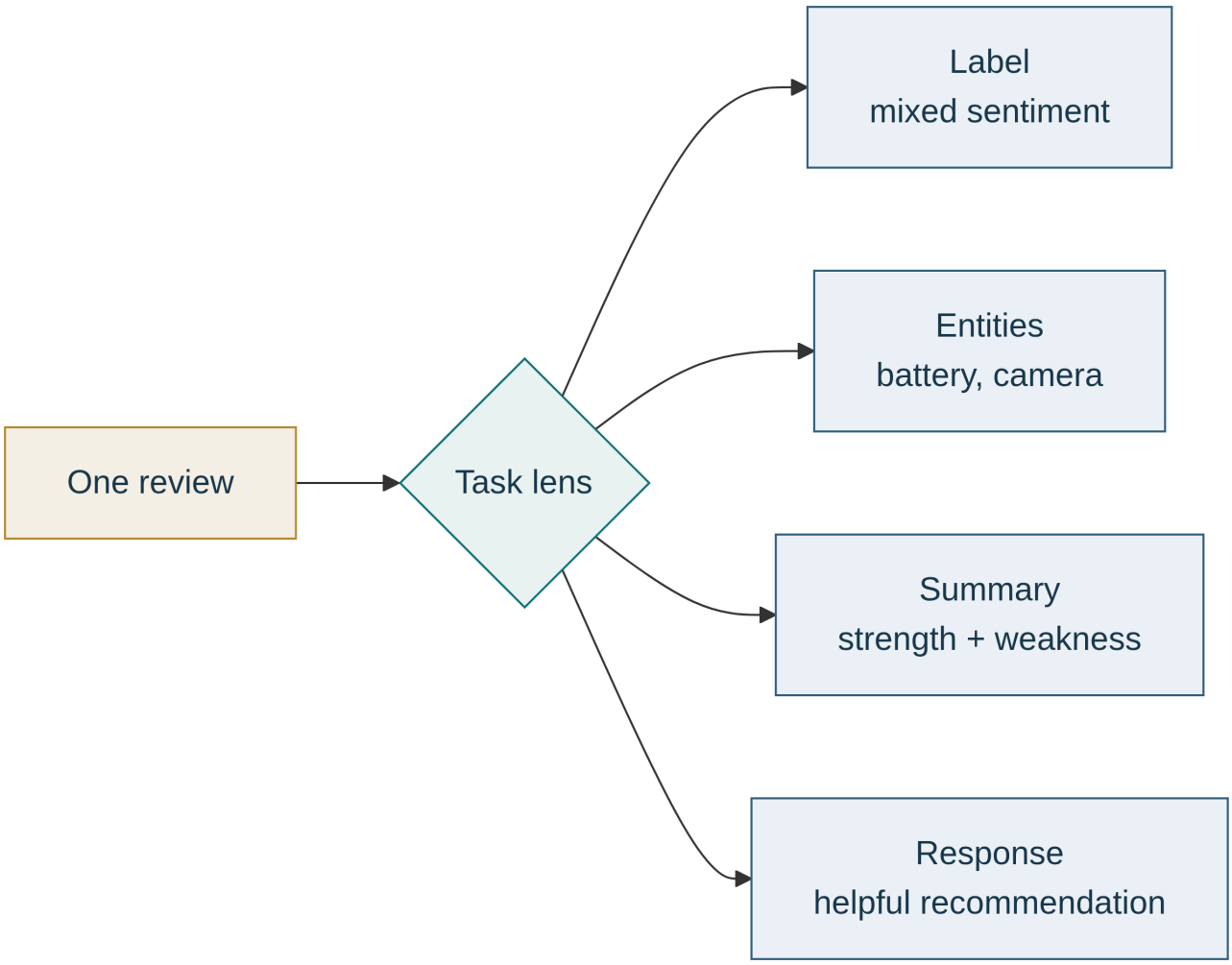


Figure 5: The task acts as a lens: the same language is transformed into different useful outputs.

3 One problem, three eras

Imagine that we must label a product review as positive or negative. The task stays fixed. What changes across NLP history is **where the useful knowledge comes from**.

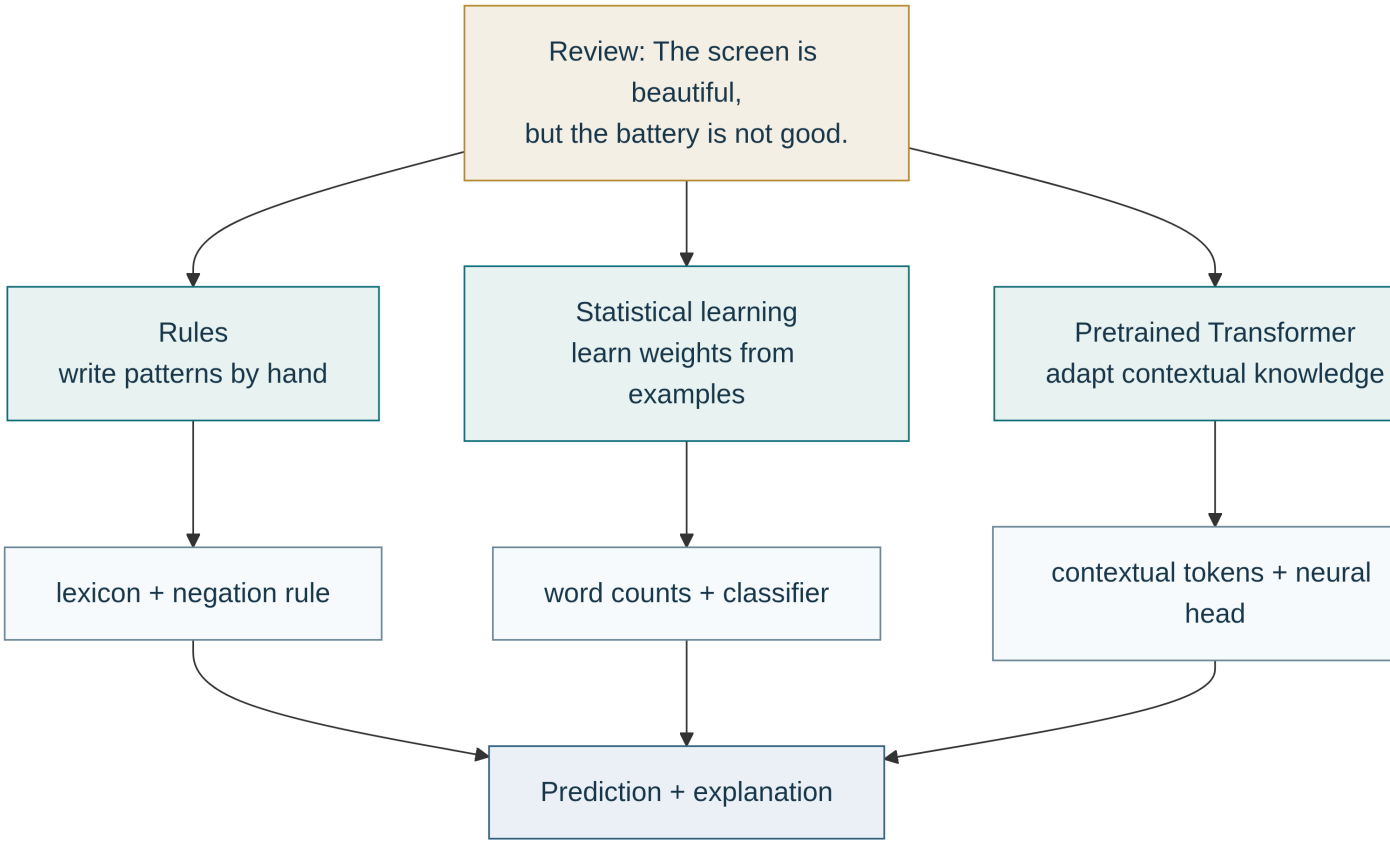


Figure 6: Three approaches to the same sentiment-classification problem.

3.1 Era I · Tell the machine the rules

A rule-based system uses knowledge written explicitly by people:

```
beautiful  → +1
good       → +1
not good   → reverse the score
```

This approach is transparent and can work well in a narrow domain. Its weakness is coverage: every new expression, exception, and context may require another rule.

3.2 Era II · Learn patterns from examples

A statistical classifier receives labeled reviews and estimates which observable features predict each label. Instead of assigning weights manually, it learns them from data.

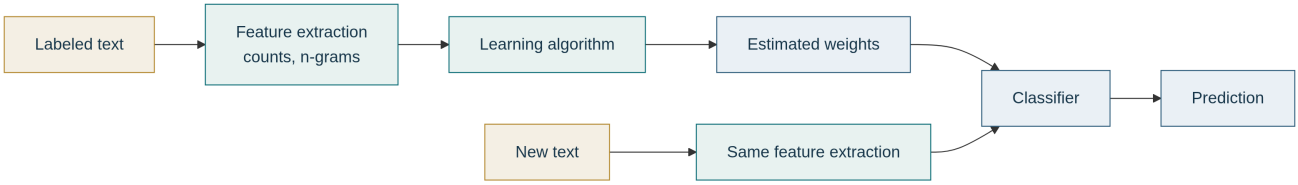


Figure 7: The classical statistical-learning recipe.

Statistical learning scales beyond hand-written rules, but its representation is often shallow. A bag of words knows that *good* occurred; it does not naturally understand why *not good* differs from *good*.

3.3 Era III · Learn representations and reuse knowledge

Neural models learn both the representation and the prediction function. Modern Transformers go further: they first learn from enormous collections of text, then reuse that knowledge for a particular task.

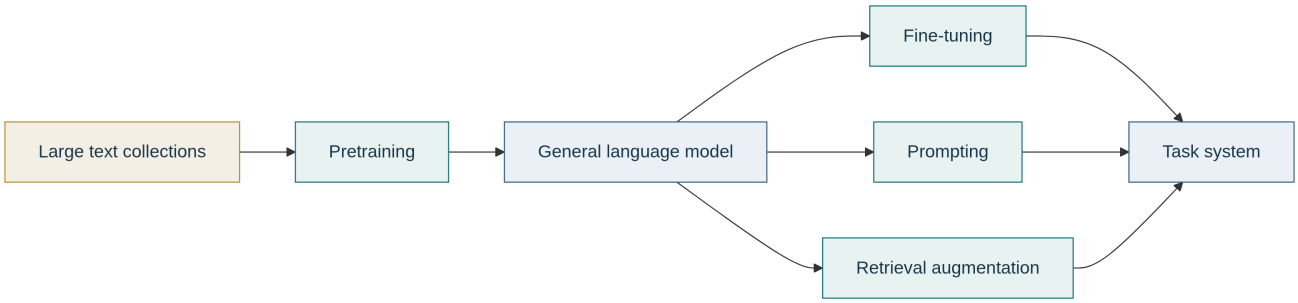


Figure 8: Modern NLP separates broad pretraining from task-specific adaptation or prompting.

The newer method is not automatically the best choice. A small rule system may be cheaper, easier to audit, and more reliable for a stable narrow task. The right question is not “Which era won?” but **which assumptions and trade-offs fit the problem?**

Question	Rules	Statistical learning	Pretrained Transformers
Where does knowledge come from?	Human experts	Labeled examples	Pretraining data + task data/instructions
Representation	Designed by hand	Usually engineered features	Learned contextual representations
Data requirement	Low	Moderate labeled dataset	Large pretraining; adaptation can use less labeled data
Interpretability	Often high	Often manageable	Usually more difficult
Compute cost	Low	Low to moderate	Moderate to very high

Question	Rules	Statistical learning	Pretrained Transformers
Typical strength	Control in narrow domains	Strong, efficient baselines	Transfer and broad language capability

Project 01 · The NLP Time Machine

Your first coding project turns this comparison into an experiment. You will send the same sentiment examples through rules, a statistical classifier, and a pretrained Transformer—then analyze where each system fails.
Open Project 01

4 Breakthroughs that changed the field

NLP did not jump directly from dictionaries to chatbots. Each breakthrough removed one bottleneck and exposed another.

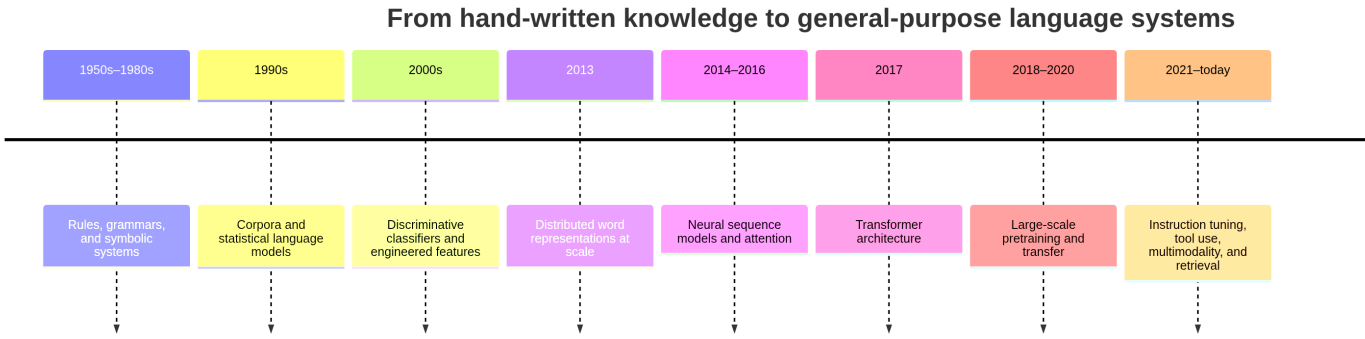


Figure 9: A conceptual timeline of major shifts in NLP. Dates mark influential milestones, not clean boundaries between eras.

4.1 The recurring pattern

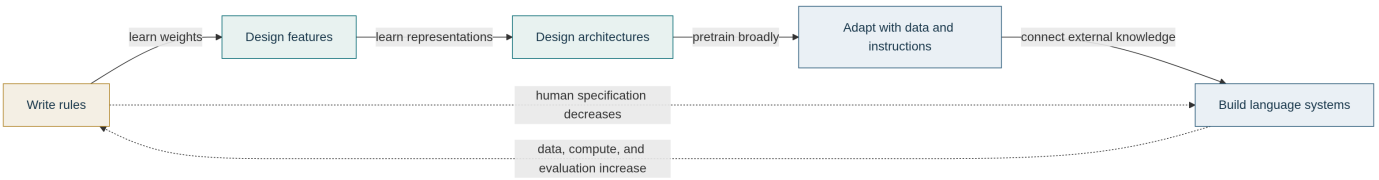


Figure 10: Breakthroughs repeatedly move work from explicit human design into learning, while increasing the importance of data and evaluation.

4.1.1 Statistical language models

Counting sequences of words made uncertainty measurable. A model could assign probabilities to sentences, rank alternatives, and generate text—although it struggled with rare phrases and long-range context.

4.1.2 Distributed representations

Word embeddings replaced isolated symbols with vectors learned from usage. Similar contexts produced nearby representations, giving systems a geometric notion of relatedness.

4.1.3 Neural sequence models

Recurrent networks processed text in order and learned features jointly with the task. They improved sequence modeling but remained difficult to train over long contexts.

4.1.4 Attention

Attention allowed a model to focus directly on relevant parts of an input instead of compressing everything into a single fixed-size state.

4.1.5 Transformers

Transformers made attention the central mechanism and enabled much more parallel training. Their scalability helped make broad pretraining practical (Vaswani et al. 2017).

4.1.6 Pretraining and instruction following

A pretrained model can reuse patterns learned from broad text collections. Fine-tuning, prompting, and instruction tuning turn that general model toward specific behaviors.

4.1.7 Retrieval and tools

Modern systems can search external collections, cite retrieved evidence, call software tools, and combine language with images or audio. The language model becomes one component in a larger system.

> ⚠ **Capability is not understanding, and fluency is not evidence**
>
> Modern models can produce convincing language while being wrong, biased, outdated, or unsupported. Throughout the course, evaluation and error analysis will matter as much as model architecture.

5 The questions that follow us

Every module in this course returns to four questions.

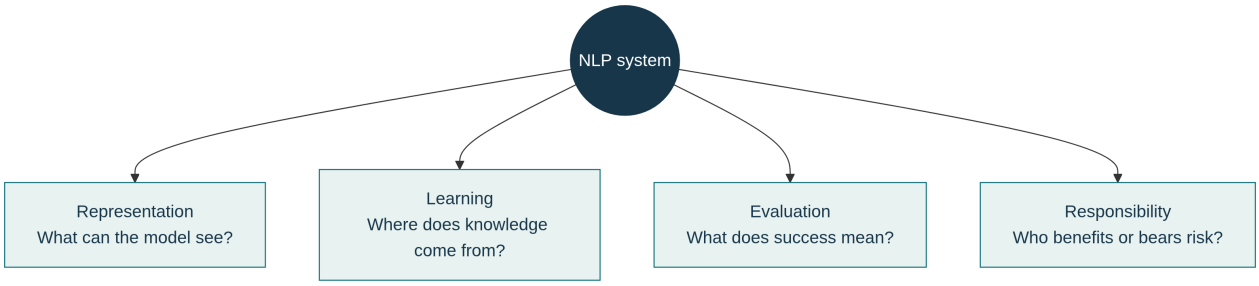


Figure 11: Four questions for analyzing any NLP system.

These questions prevent us from treating a model as a mysterious object with a single accuracy number. They connect technical design to evidence, cost, users, and consequences.

6 Next: how does text enter a model?

We have spoken casually about giving a model "text." A computer never receives words or meanings directly. It receives encoded symbols that must be divided into units and mapped to numbers.

Consider these strings:

```
don't     do not     DON’T
résumé    resume     résumé
□□□□□!    □□□      #NaturalLanguageProcessing
```

Where are the boundaries? Which differences matter? What should remain unchanged? Even counting "characters" is less obvious than it first appears.

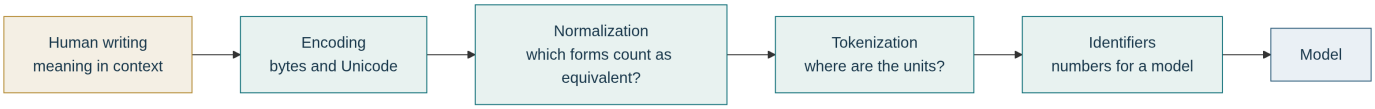


Figure 12: The next chapter opens the first black box in every NLP pipeline.

> **! The question for Chapter 2**
>
> Before a model can learn from language, **what exactly does it receive?**

The answer begins with Unicode, normalization, and tokenization.

Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. "Attention Is All You Need." *Advances in Neural Information Processing Systems* 30. https://arxiv.org/abs/1706.03762.